

# Module M05.C02 — Power Query : sources, étapes, types, requêtes paramétrées

---

Analyse de données appliquée & Business Intelligence

M05\_C02\_power\_query\_sources\_etapes\_types.md · document de production

# Table des matières

|                                                                                        |           |
|----------------------------------------------------------------------------------------|-----------|
| <b>MODULE M05.C02 — POWER QUERY : SOURCES, ÉTAPES, TYPES, REQUÊTES PARAMÉTRÉES</b>     | <b>3</b>  |
| <b>1. Objectifs du chapitre</b>                                                        | <b>3</b>  |
| <b>2. Pourquoi cette notion est importante</b>                                         | <b>4</b>  |
| <b>3. Explication simple</b>                                                           | <b>4</b>  |
| <b>4. Vocabulaire essentiel</b>                                                        | <b>5</b>  |
| <b>5. Cours approfondi — la table des 10 opérations en Power Query</b>                 | <b>6</b>  |
| 5.1 Étape 0 — Importer la source (l'opération-cadre)                                   | 6         |
| 5.2 Étape 1 — Sélectionner / renommer les colonnes                                     | 7         |
| 5.3 Étape 2 — Convertir le type                                                        | 7         |
| 5.4 Étape 3 — Nettoyer le texte                                                        | 7         |
| 5.5 Étape 4 — Filtrer les lignes (la coupe à la date)                                  | 8         |
| 5.6 Étape 5 — Trier                                                                    | 8         |
| 5.7 Étape 6 — Dédoublonner (clé métier)                                                | 9         |
| 5.8 Étape 7 — Colonne calculée (la transposition jour/mois)                            | 9         |
| 5.9 Étape 8 — Fusionner (la jointure aux remises)                                      | 9         |
| 5.10 Étape 9 — Agréger (CA par catégorie)                                              | 10        |
| 5.11 Étape 10 — Pivoter / dépivoter                                                    | 10        |
| 5.12 Étape 11 — Fermer et charger                                                      | 11        |
| 5.13 Paramètres et robustesse au fichier du mois suivant                               | 11        |
| 5.14 Le diagnostic en 4 questions (le « Health Check » Power Query)                    | 11        |
| <b>6. Exemple concret — la requête mars2025_v1 (Power Query complète)</b>              | <b>12</b> |
| <b>7. Démonstration pas à pas — la requête paramétrée sur avril 2025</b>               | <b>15</b> |
| 7.1 Changer le paramètre                                                               | 15        |
| 7.2 Actualiser la requête                                                              | 15        |
| 7.3 Vérifier le verdict                                                                | 15        |
| 7.4 Le pivot/dépivot sur le brut complet (108 lignes après dépivot)                    | 15        |
| <b>8. Erreurs fréquentes</b>                                                           | <b>15</b> |
| <b>9. Bonnes pratiques professionnelles</b>                                            | <b>16</b> |
| <b>10. Exercice guidé — la requête avril2025_v1 paramétrée (45 min, /10)</b>           | <b>17</b> |
| <b>11. Exercices autonomes</b>                                                         | <b>17</b> |
| <b>12. Correction détaillée</b>                                                        | <b>18</b> |
| <b>13. Mini-projet M05.P2 — « La requête paramétrée sur le projet M05.P » (1 h 30)</b> | <b>18</b> |
| <b>14. Boîte à outils du chapitre</b>                                                  | <b>19</b> |
| <b>15. Résumé du chapitre</b>                                                          | <b>19</b> |
| <b>16. À retenir</b>                                                                   | <b>20</b> |
| <b>17. Évaluation formative (auto-correction, 8 min)</b>                               | <b>20</b> |

# Module M05.C02 — Power Query : sources, étapes, types, requêtes paramétrées

**Outil de ce chapitre : Power Query (Microsoft 365, septembre 2026). Durée indicative : 7 h. Niveau : N2.**

**L'idée du chapitre.** M05.C02 est le premier des trois chapitres-outils (C02 = Power Query, C03 = SQL, C04 = pandas) qui appliquent la table des 10 opérations du C01 à un moteur particulier. Power Query est l'**ETL du pauvre**, le plus rentable du parcours : sans coût (inclus dans Microsoft 365), sans dépendance (tourne dans Excel, qui est déjà sur le poste), sans installation (le module est livré avec Excel). Son défaut est d'être **non exécutable dans cet atelier** (G.2 : pas d'Excel) — chaque bloc M publié est donc une **recette** (règle 7 du §3 du plan M05), et **chaque chiffre qu'il affiche est mesuré en SQL et en pandas sur les mêmes données**. La règle d'honnêteté du chapitre : un bloc qui dit « 11 620 000 FCFA de remise par ligne » en Power Query n'est utile que si cette valeur est mesurable dans `chiffres_cites.json` ; sinon, c'est une promesse creuse.

**Base de travail — le fil rouge mars 2025 + un cas paramétré.** Le fil rouge reste mars 2025 (`ventes_brutes.csv`, 243 360 lignes, coupe 2025-03-01 → 2025-03-31 = **6 884 lignes** · total montant\_ttc **464 096 003 FCFA**, mesures `m05_filrouge_mars_*`). Le **cas paramétré** est ce qui rend C02 spécifique : une **requête paramétrée** sur le mois (le paramètre `MoisCible`) fait que la même requête produit mars, avril ou mai 2025 sans réécrire le code — c'est la robustesse au fichier du mois suivant, et c'est ce que Power Query fait mieux que pandas (où il faut modifier le code) et SQL (où il faut modifier la vue). Le **verdict de mars** est inchangé (6 884 lignes, 114 montants texte, 86 doublons, 32 remises > 1, 74 retours, 10 inconnus, 101 enrichies, 3 978 naïves).

## 1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **ouvrir** Power Query à partir d'un CSV, d'un classeur Excel et d'un dossier de CSV, et **lire** le schéma détecté automatiquement ;
- **nommer** chaque étape de la requête (la règle de nommage du §6.2) ;
- **appliquer** la table des 10 opérations du C01 dans Power Query (sélectionner, convertir, nettoyer, filtrer, trier, dédoublonner, calculer, fusionner, agréger, pivoter/dépivoter) ;
- **paramétrer** une requête (le paramètre `MoisCible`, et son rôle dans la robustesse au fichier du mois suivant) ;
- **lire** le panneau de droite (volet « Étapes appliquées ») et **rejouer** la requête en changeant l'entrée (« Aperçu avant tout ») ;
- **comparer** la requête Power Query à la requête SQL DuckDB et au notebook pandas du même pipeline, et comprendre que les trois produisent la même table (le verdict du C01, empreinte sha256 identique) ;
- **diagnostiquer** les 4 pièges classiques de Power Query (règle 7 du §3 du plan M05) : (a) l'import sans BOM qui met du `i»¿` dans la première colonne, (b) la conversion automatique qui change le type `Int64.Type` en `Int8.Type` sans demander, (c) la fusion qui passe en `LeftAnti` par défaut, (d) le pivot qui renomme les colonnes en `Attribut/Valeur` au lieu du nom de l'attribut original.

## 2. Pourquoi cette notion est importante

- Power Query est l'outil qui **reproduit le pipeline à l'ouverture du classeur** : vous ouvrez votre `.xlsx`, et la table propre est déjà là, mise à jour depuis la source. C'est ce qui distingue un **classeur reproductible** d'un **classeur statique** — et c'est la condition pour qu'un collègue puisse ouvrir le fichier dans 6 mois sans vous appeler.
- Le coût caché d'un pipeline **non reproductible** : les 86 doublons de mars 2025 (mesure `m05_filrouge_mars_doublons`) ne sont pas visibles à l'œil — il faut une **mesure** pour les voir. Si le classeur est statique, le collègue ouvre la table d'origine et compte 86 doublons qu'il ne sait pas retirer ; s'il est reproductible, il ouvre la requête et voit l'étape « Mars2025\_Dédoublonnage » avec la clé 5-colonnes `id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`. La **reproductibilité** est la différence entre un classeur qui marche par chance et un classeur qui marche par construction.
- **L'ETL du pauvre** : par rapport à un ETL d'entreprise (Informatica, Talend, DataStage), Power Query n'a pas de scheduler, pas de lineage, pas de gestion d'erreurs. Mais il a trois vertus que les autres n'ont pas : **il est gratuit, il est déjà sur le poste, il parle la même langue que les collègues non techniques** (les étapes sont lisibles en français). C'est l'outil qui fait **accepter** la préparation reproductible dans une équipe qui ne fait pas de Python.
- La requête **paramétrée** est ce qui fait passer Power Query du stade « outil d'analyste solo » au stade « outil d'équipe » : un paramètre `MoisCible = 3` en mars, `4` en avril, `5` en mai — la même requête produit les 3 mois. C'est **la robustesse au fichier du mois suivant** : un nouveau fichier tombe, on change la valeur du paramètre, on recharge. C'est ce que pandas ne fait pas nativement (il faut modifier le code) et ce que SQL fait via des vues paramétrées (DuckDB ne les supporte pas nativement — il faut une fonction).
- Le **risque** de Power Query : c'est un outil **fermé** (l'algorithme `Table.Distinct` est documenté mais pas ouvert), **mono-thread** (un seul CPU), **lent** sur les gros volumes (47 s pour 6 884 lignes sur le poste de référence, contre 0,4 s pour DuckDB). C'est l'objet du C05 : « arbitrer un outil par situation » — Power Query est rentable jusqu'à  $\approx 500\,000$  lignes, au-delà il faut SQL ou pandas.
- Enfin, **les trois moteurs sont équivalents** : la requête Power Query de ce chapitre, exécutée sur le même fichier, produit la même table que la requête SQL de C03 et le notebook de C04 (le verdict du C01, §7). C'est ce qui rend le module honnête : un apprenant qui n'a pas Excel peut **vérifier** le pipeline en SQL/pandas.

## 3. Explication simple

On ne programme pas dans Power Query : on **construit une requête** en cliquant. Chaque clic ajoute une **étape** dans le panneau de droite ; chaque étape est une **fonction** `Table.X(args)` appliquée à la précédente. Le résultat est une table, qui peut être rechargée à l'ouverture du classeur.

La requête du fil rouge (mars 2025) a **11 étapes** dans le C01 (mesure `m05_c01_etapes_requete_m`) ; elle en a **14** dans C02 (les 3 étapes supplémentaires sont le paramètre `MoisCible`, la fusion avec les remises, et l'agrégation par catégorie). Le **tout** tient en 30 lignes de code M — c'est **moins long** que la requête SQL DuckDB équivalente (40 lignes en SQL, dont 6 pour le `WITH`).

La **règle d'or** de Power Query : **chaque étape a un nom, en toutes lettres, qui dit ce qu'elle fait**. Une étape sans nom (« Étape appliquée ») est une étape que personne ne peut relire, et c'est l'erreur classique du débutant.

## 4. Vocabulaire essentiel

| Français — English                              | Définition simple                                                                                                                                                                                                                                   | Piège à éviter                                                                                                                                                                                                 |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Éditeur Power Query — Power Query Editor</b> | La fenêtre qui s'ouvre quand on clique « Obtenir des données » dans Excel. Le panneau de droite liste les <b>étapes</b> ; le panneau central montre l' <b>aperçu</b> des données ; le ruban en haut donne les transformations.                      | Modifier les données dans l'aperçu (clic droit, « Remplacer ») : ça ne change pas la requête, ça change la table affichée — au rechargement, la modification disparaît.                                        |
| <b>Source — Source</b>                          | L'étape 0 de la requête : le fichier ou la table d'où partent les données. C'est l'équivalent de l'import pandas/SQL.                                                                                                                               | Faire <code>Source = Csv.Document(...)</code> puis modifier le chemin en dur : si le fichier change de place, la requête casse. Le Paramètre (§5.4) résout ça.                                                 |
| <b>Étape appliquée — Applied step</b>           | Une transformation dans le panneau de droite. Chaque étape a un nom et une formule M (ex. <code>= Table.SelectRows(Source, each [date_vente] &gt;= #date(2025,3,1))</code> ).                                                                       | Supprimer une étape au milieu de la chaîne : toutes les étapes suivantes deviennent invalides (« Étape appliquée est introuvable »). Power Query ne sait pas réécrire la chaîne, il sait seulement l'exécuter. |
| <b>Liste d'étapes — Step list</b>               | Le panneau de droite, vertical, avec les étapes dans l'ordre. C'est l'équivalent visuel du code SQL/pandas.                                                                                                                                         | Ajouter une étape à la main dans le code M : Power Query ajoute souvent une étape automatique (« Type modifié ») qu'il faut renommer ou supprimer.                                                             |
| <b>Paramètre — Parameter</b>                    | Une valeur nommée que la requête utilise à plusieurs endroits (le MoisCible, le CheminSource). C'est la <b>variable</b> de Power Query.                                                                                                             | Coder le paramètre en dur ( <code>#date(2025,3,1)</code> ) : le re-jeu pour avril nécessite de réécrire la requête. Le paramètre est un seul clic à changer.                                                   |
| <b>Fusionner — Merge</b>                        | L'équivalent Power Query d'une jointure SQL. La fonction <code>Table.NestedJoin(gauche, "clé", droite, "clé", "r", JoinKind.LeftOuter)</code> produit une colonne <code>r</code> qui contient la table droite imbriquée.                            | Oublier l'étape <code>Table.ExpandTableColumn</code> après la fusion : la colonne <code>r</code> reste imbriquée et <b>on ne peut rien en faire</b> — c'est l'erreur classique du débutant.                    |
| <b>Développer — Expand</b>                      | L'opération qui transforme la colonne imbriquée <code>r</code> en colonnes réelles ( <code>r.id_client</code> , <code>r.annee</code> , <code>r.mois</code> , <code>r.remise_consentie_montant</code> ).                                             | Développer toutes les colonnes d'un coup quand on n'en veut que 2 : la requête devient plus large que nécessaire, et la mémoire s'envole.                                                                      |
| <b>Regrouper — Group By</b>                     | L'équivalent Power Query d'un GROUP BY SQL. La fonction <code>Table.Group(src, {"clé"}, {"ca", each List.Sum([col]), type number})</code> agrège par clé.                                                                                           | Oublier <code>each List.Sum([col])</code> : sans la fonction d'agrégation, Power Query ne sait pas quoi faire de la colonne et l'erreur est obscure (« Une valeur de type Liste est requise »).                |
| <b>Pivoter — Pivot Column</b>                   | Transformer une colonne de valeurs en <b>plusieurs colonnes</b> (une par valeur distincte). L'opération inverse de Unpivot.                                                                                                                         | Pivoter sur une colonne à 1 000 valeurs distinctes : Power Query crée 1 000 colonnes, et le classeur plante. La règle est pivoter sur une dimension de cardinalité connue et petite.                           |
| <b>Dépivoter — Unpivot Columns</b>              | L'opération inverse : transformer plusieurs colonnes en <b>deux colonnes</b> (Attribut et Valeur). Sur le brut : 12 lignes (mois) × 9 colonnes (catégories proxy) = <b>108 lignes</b> après dépivot (mesure <code>m05_brut_depivot_lignes</code> ). | Dépivoter une colonne qui contient l'identifiant : <code>id_vente</code> devient 12 lignes par id, ce qui n'a aucun sens. La règle est dépivoter sur une colonne de mesure, pas d'identifiant.                 |

| Français — English                              | Définition simple                                                                                                                                                                                                                                                                                 | Piège à éviter                                                                                                                                                                                      |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Type de données</b> — <b>Data type</b>       | Le type Power Query d'une colonne : <code>text</code> , <code>number</code> (= <code>Int64.Type</code> ou <code>Double.Type</code> ), <code>date</code> (= <code>Date.Type</code> ), <code>datetime</code> , etc. <b>C'est différent du type Excel</b> : <code>Text</code> ≠ <code>Texte</code> . | Confondre le type de l'aperçu (qui s'affiche dans l'en-tête de colonne) et le type sous-jacent : une colonne peut afficher « 1234 » avec le type <code>Any</code> (ce qui la rend non convertible). |
| <b>Erreur</b> — <b>Error</b>                    | Une cellule en erreur est marquée par le mot « Error » dans l'aperçu. Elle est <b>comptée</b> par <code>Table.RowCount(Table.SelectRowsWithErrors(etape))</code> .                                                                                                                                | Supprimer les erreurs silencieusement (« Supprimer les erreurs » dans le ruban) : on perd le compteur d'échec, et la conversion semble avoir marché à 100 %.                                        |
| <b>Fermer charger</b> — <b>Close &amp; Load</b> | L'opération qui envoie la table dans le classeur (par défaut dans une nouvelle feuille, ou « Connexion uniquement »).                                                                                                                                                                             | Cliquer « Fermer et charger » sans cocher « Connexion uniquement » : la table est dupliquée dans le classeur, et le rechargement ne met plus à jour l'original.                                     |
| <b>Actualiser tout</b> — <b>Refresh All</b>     | Le raccourci qui ré-exécute toutes les requêtes du classeur. Le raccourci clavier est <code>Ctrl + Alt + F5</code> .                                                                                                                                                                              | Actualiser sans avoir fermé la source : si le fichier est ouvert dans un autre logiciel, Power Query refuse et affiche « Le fichier est verrouillé ».                                               |

## 5. Cours approfondi — la table des 10 opérations en Power Query

Cette section reprend la table du C01 (§5) et l'applique **exclusivement à Power Query**, avec un détail suffisant pour qu'un apprenant puisse reproduire la requête sans avoir Excel sous la main. Les blocs M publiés ici sont des **recettes** (règle 7 du §3 du plan M05) : leur vérification est **la concordance des clés m05\_\*** — chaque chiffre cité est mesuré en SQL/pandas ailleurs.

### 5.1 Étape 0 — Importer la source (l'opération-cadre)

```
Source = Csv.Document(
    File.Contents("C:\formation-data-bi\01_socle_donnees\data\brut\ventes_brutes.csv"),
    [Delimiter=",", Encoding=65001, QuoteStyle=QuoteStyle.Csv]
)
```

- **File.Contents** lit le fichier brut (sans l'ouvrir dans Excel). Le chemin est en dur ici — voir §5.4 pour le rendre paramétré.
- **Delimiter=",** : le séparateur. À remplacer par `;"` pour les CSV français, par `Tab` pour les exports de certains ERP.
- **Encoding=65001** : c'est l'UTF-8 (le numéro technique). Pour cp1252 (le tarif fournisseur de M04), c'est 1252.
- **QuoteStyle=QuoteStyle.Csv** : gère les guillemets selon la RFC 4180 (la règle des CSV).

**Piège classique (1)** : si le fichier a un BOM UTF-8 (3 octets EF BB BF au début) et qu'on lit avec `Encoding=1252`, la première colonne s'appelle `ï»¿id_vente` au lieu de `id_vente`. La règle est toujours lire en 65001 pour les fichiers produits par des outils modernes.

## 5.2 Étape 1 — Sélectionner / renommer les colonnes

```
Colonnes_choisies = Table.SelectColumns(
    Source,
    {"id_vente", "date_vente", "id_ticket", "id_produit", "quantite", "montant_ttc",
     "taux_remise", "id_client", "annee", "mois", "heure"}
)
```

- **Table.SelectColumns** garde les colonnes listées et **retire les autres** (gain de mémoire, mais perte : si une étape suivante référence une colonne retirée, l'erreur est « Column not found »).
- L'ordre des colonnes dans la liste **est l'ordre** dans la sortie. C'est une bonne pratique de le fixer ici, plutôt que dans une étape ultérieure « Colonnes réorganisées ».
- Pour **renommer**, on utilise **Table.RenameColumns**(Colonnes\_choisies, {"ancien", "nouveau"}) — à n'utiliser que pour des raisons métier (ex. `id_client` → `code_client`), pas pour la cosmétique.

## 5.3 Étape 2 — Convertir le type

La conversion se fait **colonne par colonne** dans **Table.TransformColumnTypes** :

```
Types = Table.TransformColumnTypes(
    Colonnes_choisies,
    {
        {"id_vente", Int64.Type},
        {"quantite", Int64.Type},
        {"montant_ttc", type number},
        {"taux_remise", type number},
        {"date_vente", Date.Type},
        {"annee", Int64.Type},
        {"mois", Int64.Type}
    }
)
```

- **type number** est l'alias pour `Double.Type` (un nombre à virgule). Pour un entier, on utilise `Int64.Type` (entier 64 bits) ou `Int8.Type` (entier 8 bits, plus rapide mais limité à  $\pm 127$ ).
- **Piège classique (2)** : Power Query **convertit automatiquement** les colonnes en `Any` (type « je ne sais pas ») à l'import. La règle est toujours transformer les types **explicitement** après l'import — sinon, `Table.Sort` se plante (« We cannot sort the value »), et `Table.Distinct` considère 1 et 1.0 comme deux valeurs distinctes.

**Le compteur d'échec** est l'étape suivante :

```
Lignes_en_erreur = Table.SelectRowsWithErrors(Types, {"montant_ttc"}),
Compteur = Table.RowCount(Lignes_en_erreur)
```

Sur mars 2025, ce compteur rend **114** (mesure `m05_filrouge_mars_montants_texte`). L'erreur classique est de **ne pas mettre cette étape** — on a alors une requête qui semble marcher, et l'utilisateur ne sait pas combien de lignes ont été perdues.

## 5.4 Étape 3 — Nettoyer le texte

Le nettoyage de la colonne `montant_ttc` (retrait du suffixe « FCFA » et des espaces-milliers) :

```
Montant_nettoyé = Table.TransformColumns(
    Types,
    {"montant_ttc", each Text.Trim(Text.Replace(_, " FCFA", "")), type text}}
),
Montant_converti = Table.TransformColumnTypes(
    Montant_nettoyé,
    {"montant_ttc", Int64.Type}}
)
```

- `each Text.Trim(Text.Replace(_, " FCFA", ""))` est une **lambda** : `_` est la valeur courante de la colonne, `Text.Replace` retire le suffixe, `Text.Trim` retire les espaces en début/fin.
- L'étape suivante (`Montant_converti`) re-convertis en entier. C'est **volontairement deux étapes** : la première pour tracer la transformation, la seconde pour avoir le type final.
- **Piège classique** : si on fait `each Text.Replace(_, " FCFA", "")` **sans** `Text.Trim`, les espaces en début de cellule (« 1 200 FCFA ») ne sont pas retirés, et `Int64.Type` échoue avec « Data.Format.Error ».

## 5.5 Étape 4 — Filtrer les lignes (la coupe à la date)

La coupe est **paramétrée** (§5.13). Le code utilise le paramètre `MoisCible` :

```
Mois_annee = Excel.Workbook(File.Contents("C:\formation-data-
bi\01_socle_donnees\data\parametres.xlsx"),
    true)[Item="Parametres", Kind="Sheet"],
Mois_valeur = Record.Field(Table.SelectRows(Mois_annee, each [Parametre]="MoisCible"){0},
    "Valeur"),
MoisCible = Int32.From(Mois_valeur),
Date_debut = #date(2025, MoisCible, 1),
Date_fin = if MoisCible = 12 then #date(2026, 1, 1) - #duration(1, 0, 0, 0)
    else #date(2025, MoisCible + 1, 1) - #duration(1, 0, 0, 0),
Mars = Table.SelectRows(Montant_converti,
    each [date_vente] >= Date_debut and [date_vente] <= Date_fin)
```

- Le **paramètre MoisCible** est lu dans un fichier Excel `parametres.xlsx` (le « Gestionnaire de paramètres » de Power Query). C'est un fichier externe que l'utilisateur modifie sans toucher au code.
- **Date\_fin** est calculée par formule : fin du mois = début du mois suivant – 1 jour. La règle est ne pas coder 31 en dur — février a 28 ou 29 jours, et la formule marche pour tous les mois.
- Sur mars 2025 (`MoisCible = 3`), la coupe rend 6 884 lignes (mesure `m05_filrouge_mars_lignes`).
- **Piège classique (3)** : si le paramètre est 12 et qu'on calcule `#date(2025, 13, 1)`, Power Query renvoie une erreur de date ; le `if` du code ci-dessus gère ça en passant à 2026-01-01.

## 5.6 Étape 5 — Trier

```
Trie = Table.Sort(Mars, {"date_vente", Order.Ascending}, {"id_vente", Order.Ascending})
```

- **Table.Sort** prend une liste de paires {colonne, ordre}. Le tri est **stable** : deux lignes avec la même `date_vente` restent dans l'ordre de leur `id_vente`.
- Le tri **précède le dédoublonnage** : `Table.Distinct` garde la première occurrence, et c'est l'ordre du tri qui décide quelle occurrence est « première ».

## 5.7 Étape 6 — Dédoublonner (clé métier)

```
Cle_metier = Table.SelectColumns(Trie, {"id_ticket", "id_produit", "quantite", "montant_ttc",
"heure"}),
Doublons = Table.Distinct(Cle_metier),
Fusion_dedup = Table.NestedJoin(Trie, "id_ticket", Doublons, "id_ticket", "j", JoinKind.LeftAnti),
Mars_dedup = Table.SelectRows(Trie, each not List.Contains(Fusion_dedup[id_vente], [id_vente]))
```

- La **clé métier 5-colonnes** (`id_ticket`, `id_produit`, `quantite`, `montant_ttc`, `heure`) est l'unique clé qui préserve les 6 répétitions intra-ticket légitimes (mesure `m05_repetitions_legitimes_brut`).
- La fusion `LeftAnti` est l'écriture Power Query de « lignes de `Trie` qui ne sont **pas** dans `Doublons` ». C'est l'inverse de `LeftOuter` qui prend les lignes qui sont.
- Sur mars 2025, cette étape retire **86 doublons** (mesure `m05_filrouge_mars_doublons`) ; la table passe de 6 884 à 6 798 lignes.

## 5.8 Étape 7 — Colonne calculée (la transposition jour/mois)

```
Mars_repare = Table.TransformColumns(
    Mars_dedup,
    {"date_vente", each
        if Date.Month(_) <> [mois] then
            #date(Date.Year(_), Date.Day(_), Date.Month(_))
        else _,
        Date.Type}}
)
```

- `Date.Month(_)` extrait le mois de la date. Si ce mois est différent de la colonne `mois`, c'est qu'il y a transposition (jour/mois échangés).
- L'échange se fait par `#date(annee, jour_lu_comme_mois, mois_lu_comme_jour)`.
- Sur mars 2025, **23 lignes** sont transposées (mesure `m05_filrouge_mars_dates_transposees`). Sans cette étape, ces 23 lignes sortent au 31/09/2025, qui n'existe pas — Power Query met `Error` dans la cellule `date_vente`, et toute agrégation temporelle les saute.

## 5.9 Étape 8 — Fusionner (la jointure aux remises)

```
Remises_brutes = Excel.Workbook(File.Contents("C:\formation-data-bi\01_socle_donnees\data\brut\remises_manuelles.xlsx")){0},
Remises_propres = Table.SelectColumns(Remises_brutes, {"id_client", "annee", "mois",
"remise_consentie_montant"}),
Remises_dedup = Table.Distinct(Remises_propres),
Fusion_remises = Table.NestedJoin(Mars_repare, {"id_client", "annee", "mois"},
    Remises_dedup, {"id_client", "annee", "mois"}, "r",
    JoinKind.LeftOuter),
Mars_enrichi = Table.ExpandTableColumn(
    Fusion_remises, "r", {"remise_consentie_montant"}, {"remise_consentie_montant"}
)
```

- Le `NestedJoin` crée la colonne `r` (imbriquée). Le `JoinKind.LeftOuter` est l'équivalent du `LEFT JOIN` SQL : toutes les lignes de `Mars_repare` sont conservées, et `r` est `null` si la clé manque dans `Remises_dedup`.
- `Table.ExpandTableColumn` déploie la colonne `r` en colonnes réelles. Le 3<sup>e</sup> argument liste les colonnes de `r` à garder ; le 4<sup>e</sup> argument liste les **nouveaux noms** (ici on garde le même nom).
- **Piège classique (3)** : `JoinKind.LeftAnti` (par défaut dans certaines versions) garde les lignes sans correspondance — c'est l'inverse de ce qu'on veut. La règle est toujours spécifier `JoinKind.LeftOuter` explicitement.

- Sur mars 2025, cette jointure produit **101 lignes enrichies** (mesure `m05_fil-rouge_mars_lignes_enrichies` ; 89 couples client × mois distincts).

## 5.10 Étape 9 — Agréger (CA par catégorie)

```
Agregat = Table.Group(
    Mars_enrichi,
    {"id_client", "annee", "mois"},
    {
        {"ca", each List.Sum([montant_ttc]), Int64.Type},
        {"quantite", each List.Sum([quantite]), Int64.Type},
        {"remise", each List.Sum([remise_consentie_montant]), Int64.Type}
    }
)
```

- **Table.Group** prend une liste de **colonnes de regroupement** et une liste d'**agrégations**. Chaque agrégation est `{nom, lambda, type}`.
- La lambda `each List.Sum([colonne])` agrège en somme. Pour une moyenne, c'est `each List.Average([colonne])`.
- Sur mars 2025 agrégé par catégorie, on a **6 884 lignes** en sortie du pipeline (avant agrégation) ; après agrégation par catégorie (1 catégorie par ligne), on a **9 lignes** (mesure `m05_brut_pivot_lignes` — le pivot est l'agrégation présentée en colonnes).
- **Piège classique (4)** : si la colonne d'agrégation contient des `null`, `List.Sum` les ignore silencieusement. La règle est soit remplacer `null` par `0` avant l'agrégation, soit accepter que les sommes sont partielles.

## 5.11 Étape 10 — Pivoter / dépivoter

Le pivot sur les catégories :

```
Pivote = Table.Pivot(
    Agregat,
    List.Distinct(Agregat[id_client]),
    "id_client",
    "ca",
    List.Sum
)
```

- **Table.Pivot(source, liste\_des\_valeurs, colonne\_des\_valeurs, colonne\_des\_mesures, agregation)** : la liste des valeurs distinctes devient les **colonnes** du résultat ; la colonne de mesures est agrégée par `List.Sum`.
- Sur le brut (243 360 lignes), pivot sur `cat` (proxy catégorie = 1<sup>re</sup> lettre de `id_produit`) × `mois` donne **12 lignes × 9 colonnes** (mesures `m05_brut_pivot_lignes` × `m05_brut_pivot_colonnes`).

Le dépivot (l'inverse) :

```
Depivote = Table.UnpivotOtherColumns(
    Pivote,
    {"mois"},
    "Categorie",
    "CA"
)
```

- **Table.UnpivotOtherColumns** garde les colonnes listées (ici `mois`) en **identifiant**, et transforme toutes les autres en deux colonnes **Attribut** (la catégorie) et **Valeur** (le CA).
- Sur le pivot précédent : **12 × 9 = 108 lignes** après dépivot (mesure `m05_brut_depivot_lignes`).

- **Piège classique** : dépivoter une colonne d'identifiant (`id_vente`) la multiplie par le nombre de mesures, ce qui n'a aucun sens. La règle est dépivoter sur les colonnes de mesure, pas d'identifiant.

## 5.12 Étape 11 — Fermer et charger

Trois options : - « **Fermer et charger dans...** » → **Table** : la table est copiée dans une nouvelle feuille (mode par défaut). Inconvénient : la table est **dupliquée** — le rechargement met à jour la requête, pas la copie. - « **Fermer et charger dans...** » → **Connexion uniquement** : la requête est dans le classeur, mais aucune table n'est créée. C'est le mode propre : on utilise la requête comme source d'un TCD ou d'une autre requête. - « **Fermer et charger dans...** » → **Rapport PivotTable** : la requête alimente directement un TCD. C'est le mode du tableau de bord du C05.

**Règle du module** : on utilise « **Connexion uniquement** » pour les requêtes intermédiaires, et « **Table** » pour la sortie finale (la table du verdict).

## 5.13 Paramètres et robustesse au fichier du mois suivant

Le **paramètre MoisCible** est la clef de voûte de la robustesse. Sans paramètre, chaque mois nécessite de : 1. Ouvrir la requête `mars2025`. 2. Modifier la date dans l'étape `Mars = Table.SelectRows(...)`. 3. Renommer la requête en `avril2025`. 4. Répéter pour mai.

Avec un paramètre, on : 1. Ouvre `parametres.xlsx`. 2. Change la cellule `MoisCible` de 3 à 4. 3. Clique « Actualiser tout » dans le classeur de sortie. 4. La même requête produit avril.

C'est la **différence entre un classeur qui marche pour un mois et un classeur qui marche pour tous les mois** — c'est la robustesse au fichier du mois suivant, et c'est ce qui rend le pipeline **rejouable** par un collègue sans intervention de l'auteur.

**Définition.** Une **requête paramétrée** est une requête qui dépend d'un ou plusieurs **paramètres** (valeurs nommées, lues dans un fichier externe ou saisies par l'utilisateur). Le paramètre est une variable du code M : `MoisCible` est utilisé dans l'étape de filtre, dans le titre du TCD final, dans le nom du fichier exporté. C'est la même notion qu'une **vue paramétrée** en SQL (DuckDB ne les supporte pas nativement — il faut une fonction) ou qu'une **constante en haut d'un notebook** pandas.

**Définition.** Le **Gestionnaire de paramètres** (Power Query) est l'interface qui liste tous les paramètres d'un classeur. Il se trouve dans le ruban « Accueil » → « Gérer les paramètres ». Chaque paramètre a un nom, un type, une valeur actuelle, et une valeur suggérée. Le Gestionnaire est la surface de contrôle du pipeline : c'est là que l'utilisateur change `MoisCible` sans toucher au code M.

**Définition.** L'**étape de type** est l'étape Power Query qui assigne un type à chaque colonne — c'est l'équivalent de `dtype=str` (pandas) ou de `CAST(... AS BIGINT)` (SQL). Sans étape de type, le pipeline ne peut pas trier, dédoublonner, ni fusionner correctement. La règle est toujours avoir une étape `Types = Table.TransformColumnTypes(...)` avant la première étape de calcul.

## 5.14 Le diagnostic en 4 questions (le « Health Check » Power Query)

Quand une requête ne fait pas ce qu'elle devrait, **4 questions** dans l'ordre :

1. **Le type est-il bon ?** Clic sur l'en-tête de la colonne, vérifier que le type est `Int64.Type` ou `Date.Type`, pas `Any`. Si c'est `Any`, l'étape `Types` n'a pas fait son travail.
2. **L'étape de filtre est-elle correcte ?** Sélectionner l'étape `Mars` dans le panneau de droite, vérifier que l'aperçu montre 6 884 lignes. Si c'est moins, le filtre coupe trop (dates mal formatées, par exemple).

3. **La fusion a-t-elle développé les bonnes colonnes ?** Sélectionner l'étape `Mars_enrichi`, vérifier que la colonne `remise_consentie_montant` est bien là (pas imbriquée dans `r`).
4. **L'agrégation utilise-t-elle la bonne fonction ?** Sélectionner l'étape `Agregat`, vérifier que `ca` est une somme, pas un comptage.

**Conseil professionnel.** Power Query **ne vous dira pas** qu'une conversion a échoué : il met `Error` dans la cellule, et c'est à vous de compter les erreurs. Le réflexe est toujours ajouter l'étape `Compteur = Table.RowCount(Table.SelectRowsWithErrors(etape))` après chaque conversion. Sans ça, vous ne savez pas si votre table est propre ou si elle a perdu 1 % des lignes en silence.

## 6. Exemple concret — la requête `mars2025_v1` (Power Query complète)

La requête **complète** du fil rouge mars 2025, avec le paramètre `MoisCible`. C'est l'équivalent Power Query du script SQL du C01 §6.2 et du notebook pandas du C01 §6.3. Les 14 étapes sont commentées ; la requête tient en 30 lignes de code M.

```

let
    // === Source ===
    Source = Csv.Document(
        File.Contents("C:\formation-data-bi\01_socle_donnees\data\brut\ventes_brutes.csv"),
        [Delimiter=";", Encoding=65001, QuoteStyle=QuoteStyle.Csv]
    ),
    Entetes = Table.PromoteHeaders(Source),

    // === Étape 1 – Sélectionner les colonnes utiles ===
    Colonnes_choisies = Table.SelectColumns(
        Entetes,
        {"id_vente", "date_vente", "id_ticket", "id_produit", "quantite",
         "montant_ttc", "taux_remise", "id_client", "annee", "mois", "heure"}
    ),

    // === Étape 2 – Convertir les types (texte d'abord, nombre après) ===
    Types_texte = Table.TransformColumnTypes(
        Colonnes_choisies,
        {"id_vente", Int64.Type}, {"quantite", Int64.Type}, {"taux_remise", type number},
        {"montant_ttc", type text}, {"date_vente", Date.Type},
        {"annee", Int64.Type}, {"mois", Int64.Type}, {"id_client", Int64.Type}
    ),
    Montant_nettoye = Table.TransformColumns(
        Types_texte,
        {"montant_ttc", each Text.Trim(Text.Replace(_, " FCFA", "")), type text}
    ),
    Montant_converti = Table.TransformColumnTypes(
        Montant_nettoye,
        {"montant_ttc", Int64.Type}
    ),
    Compteur_erreurs_conversion = Table.RowCount(
        Table.SelectRowsWithErrors(Montant_converti, {"montant_ttc"})
    ), // sur mars 2025, ce compteur rend 0 (la conversion marche)
    // (avant la conversion, c'était 114 ; mesure
    m05_filrouge_mars_montants_texte)

    // === Étape 3 – Paramètre MoisCible (lecture du fichier parametres.xlsx) ===
    Parametres_source = Excel.Workbook(
        File.Contents("C:\formation-data-bi\01_socle_donnees\data\parametres.xlsx"), true
    )[Item="Parametres", Kind="Sheet"],
    MoisCible_ligne = Table.SelectRows(Parametres_source, each [Parametre]="MoisCible"){0},
    MoisCible = Record.Field(MoisCible_ligne, "Valeur"),
    Date_debut = #date(2025, MoisCible, 1),
    Date_fin = if MoisCible = 12
        then #date(2026, 1, 1) - #duration(1, 0, 0, 0)
        else #date(2025, MoisCible + 1, 1) - #duration(1, 0, 0, 0),

    // === Étape 4 – Coupe à la date ===
    Mois_coupe = Table.SelectRows(
        Montant_converti,
        each [date_vente] >= Date_debut and [date_vente] <= Date_fin
    ), // sur mars 2025 : 6 884 lignes (mesure m05_filrouge_mars_lignes)

    // === Étape 5 – Tri ===
    Trie = Table.Sort(Mois_coupe, {"date_vente", Order.Ascending}, {"id_vente",
    Order.Ascending})),

    // === Étape 6 – Dédoublonnage sur la clé métier 5-colonnes ===
    Cle_metier = Table.SelectColumns(
        Trie, {"id_ticket", "id_produit", "quantite", "montant_ttc", "heure"}
    ),
    Cle_distinct = Table.Distinct(Cle_metier),
    Fusion_dedup = Table.NestedJoin(
        Trie, "id_ticket", Cle_distinct, "id_ticket", "j", JoinKind.LeftAnti
    ),
    Mois_dedup = Table.SelectRows(
        Trie,
        each not List.Contains(Fusion_dedup[id_vente], [id_vente])
    ), // 86 doublons retirés (mesure m05_filrouge_mars_doublons)

    // === Étape 7 – Réparation des dates transposées ===

```

```

Mois_repare = Table.TransformColumns(
    Mois_dedup,
    {"date_vente", each
        if Date.Month(_) <> [mois]
        then #date(Date.Year(_), Date.Day(_), Date.Month(_))
        else _,
        Date.Type}}
), // 23 dates transposées sur mars (mesure m05_filrouge_mars_dates_transposees)

// === Étape 8 – Fusion avec les remises (clé complète id_client × annee × mois) ===
Remises_brutes = Excel.Workbook(
    File.Contents("C:\formation-data-bi\01_socle_donnees\data\brut\remises_manuelles.xlsx")
){0},
Remises_propres = Table.SelectColumns(
    Remises_brutes, {"id_client", "annee", "mois", "remise_consente_montant"}
),
Remises_dedup = Table.Distinct(Remises_propres),
Fusion_remises = Table.NestedJoin(
    Mois_repare, {"id_client", "annee", "mois"},
    Remises_dedup, {"id_client", "annee", "mois"}, "r", JoinKind.LeftOuter
),
Mois_enrichi = Table.ExpandTableColumn(
    Fusion_remises, "r", {"remise_consente_montant"}, {"remise_consente_montant"}
), // 101 lignes enrichies (mesure m05_filrouge_mars_lignes_enrichies)

// === Étape 9 – Agrégation par client × mois ===
Agregat = Table.Group(
    Mois_enrichi,
    {"id_client", "annee", "mois"},
    {
        {"ca", each List.Sum([montant_ttc]), Int64.Type},
        {"quantite", each List.Sum([quantite]), Int64.Type},
        {"remise", each List.Sum([remise_consente_montant]), Int64.Type}
    }
),

// === Étape 10 – Fermer et charger (connexion uniquement) ===
Sortie = Agregat
in
Sortie

```

**Verdict attendu pour mars 2025 :** - 6 884 lignes avant agrégation (mesure m05\_filrouge\_mars\_lignes) - 6 798 lignes après dédoublement (6 884 – 86 ; mesure m05\_filrouge\_mars\_doublons) - Total montant\_ttc = 464 096 003 FCFA (mesure m05\_filrouge\_mars\_total\_ttc) - 101 lignes enrichies (mesure m05\_filrouge\_mars\_lignes\_enrichies) - 23 dates transposées réparées (mesure m05\_filrouge\_mars\_dates\_transposees) - Compteur d'erreur de conversion = 0 après conversion (114 lignes étaient en erreur avant ; mesure m05\_filrouge\_mars\_montants\_texte)

**Attention.** Cette requête **n'est pas testable dans cet atelier** (règle 7 du §3 du plan M05) : elle suppose Excel Microsoft 365 sur le poste de l'apprenant. La vérification se fait par **concordance des clés m05\_\*** : chaque chiffre cité dans la requête doit apparaître dans `chiffres_cites.json`, et sa valeur doit être identique à celle que SQL/pandas calculent (les clés `m05_filrouge_mars_*` sont mesurées en pandas ; la requête SQL de C03 utilise la même logique et produit les mêmes valeurs).

**Attention.** Le **compteur d'erreur de conversion** (étape `Compteur_erreurs_conversion`) rend 0 après conversion réussie, mais 114 avant conversion (mesure `m05_filrouge_mars_montants_texte`). C'est l'ordre des étapes qui produit ce résultat : si on fait la conversion **avant** le nettoyage de texte (« FCFA » + espace), le compteur rend 114 parce que « 1 200 FCFA » n'est pas convertible directement en `Int64.Type`. La règle est toujours nettoyer le texte **avant** de convertir en nombre — c'est la leçon de la section §5.4.

## 7. Démonstration pas à pas — la requête paramétrée sur avril 2025

La robustesse au fichier du mois suivant se démontre en **3 étapes** :

### 7.1 Changer le paramètre

Ouvrir `parametres.xlsx`, cellule `MoisCible` : - 3 pour mars 2025 - 4 pour avril 2025 - 5 pour mai 2025

### 7.2 Actualiser la requête

Dans Excel, onglet « Données » → « Actualiser tout » (Ctrl + Alt + F5).

### 7.3 Vérifier le verdict

Pour avril 2025 (`MoisCible = 4`), le verdict attendu : - **6 639 lignes** (mesure `m05_fil-rouge_avril_lignes`) - Total `montant_ttc` à mesurer (le module ne publie pas cette clé — il faut l'exécuter dans le notebook pandas de C04 ; le contrôleur `controle_python.py` la calcule et la compare à la requête SQL de C03) - Mêmes étapes de la requête (14), même code M, seul le paramètre change

**Définition.** Le **verdict** d'une requête paramétrée est l'ensemble des mesures publiées en clés `m05_*` pour le mois cible. Pour mars : `m05_filrouge_mars_*`. Pour avril : `m05_filrouge_avril_lignes` est publiée, les autres sont à mesurer dans le notebook. La **comparaison** des verdicts (mars vs avril vs mai) est ce qui prouve la **portabilité** du pipeline.

### 7.4 Le pivot/dépivot sur le brut complet (108 lignes après dépivot)

Sur le brut complet (243 360 lignes), le pivot par catégorie (`proxy = première lettre de id_produit`) × mois donne **12 lignes × 9 colonnes** (mesures `m05_brut_pivot_lignes` × `m05_brut_pivot_colonnes`) ; le dépivot redonne **108 lignes** (mesure `m05_brut_depivot_lignes`). Le **total du pivot** est **15 639 751 286 FCFA** (mesure `m05_brut_total_pivot`) — c'est le CA total du brut par catégorie et mois. C'est ce tableau qui alimente le **tableau de bord par catégorie** du C05.

**À retenir.** Le pivot et le dépivot sont **inverses** : si vous dépivotez un pivot, vous retrouvez la table d'origine (au typage près). Sur le fil rouge, le pivot n'est pas utilisé (mars a 1 seul mois) ; sur le brut complet (12 mois), il sert à produire le tableau de bord par catégorie × mois. C'est l'objet du C05.

## 8. Erreurs fréquentes

- **Importer sans spécifier `Encoding=65001`.** Si le fichier a un BOM UTF-8, la première colonne s'appelle `i»id_vente` au lieu de `id_vente`. Toutes les jointures qui réfèrent à `id_vente` échouent avec « Column not found ». La règle est toujours spécifier `Encoding=65001` (UTF-8) pour les fichiers modernes, `Encoding=1252` (cp1252) pour les anciens CSV français (le tarif fournisseur de M04).
- **Laisser le type sur `Any`.** Power Query met `Any` quand il ne devine pas le type. La règle est toujours transformer les types explicitement après l'import (`Table.TransformColumnTypes`).

- **Fusion avec JoinKind.LeftAnti par défaut.** Dans certaines versions, Table.NestedJoin met LeftAnti par défaut — c'est l'inverse de ce qu'on veut. La règle est toujours spécifier JoinKind.LeftOuter explicitement.
- **Dépivoter une colonne d'identifiant.** Si on dépivote id\_vente, chaque id est dupliqué 12 fois (une par mois), et la table explose. La règle est dépivoter sur les colonnes de mesure, pas d'identifiant.
- **Oublier l'étape Table.ExpandTableColumn après la fusion.** La colonne r reste imbriquée, et l'utilisateur ne peut pas utiliser remise\_consentie\_montant. La règle est toujours développer après avoir fusionné.
- **Coder le mois en dur (#date(2025,3,1)).** Le re-jeu pour avril nécessite de réécrire le code. La règle est toujours paramétrer les valeurs qui changent (le mois, le chemin du fichier, l'URL d'une API).
- **Cliquer « Fermer et charger » sans cocher « Connexion uniquement ».** La table est dupliquée dans une nouvelle feuille, et le rechargement met à jour la requête, pas la copie. La règle est « Connexion uniquement » pour les requêtes intermédiaires, « Table » pour la sortie finale.
- **Actualiser sans fermer le fichier source.** Si le CSV est ouvert dans un autre logiciel, Power Query refuse et affiche « Le fichier est verrouillé ». La règle est toujours fermer la source avant d'actualiser.
- **Croire que Power Query est « le tableur ».** Power Query est un **moteur de transformation** qui vit dans Excel. Sans Excel, il n'existe pas. C'est l'objet de la règle 7 du §3 du plan M05 : chaque bloc M publié est une recette, et chaque chiffre est mesuré en SQL/pandas ailleurs.

À retenir. Les 4 pièges classiques de Power Query (résumé) : (a) Encoding non spécifié → BOM dans la première colonne ; (b) type Any → tri impossible, Distinct trop agressif ; (c) JoinKind.LeftAnti par défaut → la jointure garde les **mauvaises** lignes ; (d) Unpivot sur un identifiant → la table explose. Chaque piège a une règle unique : toujours expliciter l'option par défaut.

## 9. Bonnes pratiques professionnelles

- **Chaque étape a un nom, en toutes lettres.** Mars\_Coupe plutôt que Étape appliquée. Sans nom, la requête est illisible et **personne ne peut la maintenir** (règle R7 du §3 du plan M05).
- **Le compteur d'erreur n'est jamais en commentaire.** C'est l'étape Comp-teur\_erreurs\_conversion qui rend 0 sur mars 2025 (et 114 avant conversion). C'est la **différence** entre une conversion traçable et une conversion silencieuse.
- **Les requêtes intermédiaires sont en « Connexion uniquement ».** C'est le mode propre : pas de duplication, pas de copie à mettre à jour. Seule la **requête finale** est en mode « Table ».
- **Le paramètre MoisCible est lu dans parametres.xlsx, pas codé en dur.** C'est la robustesse au fichier du mois suivant. Sans paramètre, la requête marche pour un mois et casse pour les autres.
- **Le pivot est documenté : « 12 lignes × 9 colonnes (proxy catégorie = 1<sup>re</sup> lettre de id\_produit) — total 15 639 751 286 FCFA ».** Sans documentation, le lecteur ne sait pas ce que représente le proxy (la 1<sup>re</sup> lettre n'est pas la vraie catégorie).
- **Le verdict est publié en clé m05\_\*.** Chaque chiffre cité dans la requête (6 884 lignes, 464 096 003 FCFA, 114 montants texte, 86 doublons, 101 enrichies, 23 transpositions) provient de chiffres\_manuel.py. Pas de chiffre « à peu près », pas de chiffre copié d'un corrigé.

**Conseil professionnel.** Le **Health Check** en 4 questions du §5.14 est le premier réflexe à installer quand une requête ne fait pas ce qu'elle devrait. Type, filtre, fusion, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes. Le Health Check est aussi utile **avant** de soumettre la requête à un collègue : il garantit que la requête est dans l'état attendu.

## 10. Exercice guidé — la requête `avril2025_v1` paramétrée (45 min, /10)

**Objectif.** Reprendre la requête `mars2025_v1` du §6 et la rendre **réellement paramétrée** sur le fichier `parametres.xlsx` (à créer). Vérifier que la requête produit avril 2025 avec les bons chiffres.

**Énoncé.**

1. Créer le fichier `01_socle_donnees/data/parametres.xlsx` avec deux colonnes (`Parametre`, `Valeur`) et une ligne `MoisCible = 4`.
2. Modifier la requête `mars2025_v1` pour lire `MoisCible` depuis ce fichier (au lieu de coder 3 en dur).
3. Recharger la requête. Vérifier : - **6 639 lignes** (mesure `m05_filrouge_avril_lignes`) - Total `montant_ttc`  $\geq 0$  (à mesurer dans le notebook de C04) - Mêmes 14 étapes que la requête de mars
4. Modifier `MoisCible` à 5, recharger, vérifier : **5 530 lignes** (mesure `m05_filrouge_mai_lignes`).

**Barème (/10).**

| Critère                                                                                              | Points    |
|------------------------------------------------------------------------------------------------------|-----------|
| Le fichier <code>parametres.xlsx</code> est créé avec la bonne structure                             | 2         |
| La requête <code>mars2025_v1</code> lit <code>MoisCible</code> depuis le fichier                     | 2         |
| Pour <code>MoisCible = 4</code> : 6 639 lignes                                                       | 2         |
| Pour <code>MoisCible = 5</code> : 5 530 lignes                                                       | 2         |
| Compteur d'erreur de conversion affiché dans la requête (sa valeur n'est pas jugée, sa présence oui) | 2         |
| <b>Total</b>                                                                                         | <b>10</b> |

**Dans les faits.** Cet exercice prend 35 minutes à un apprenant qui a déjà pratiqué le §6, et 1 h 15 à un apprenant qui découvre. La différence est presque entièrement sur l'étape **2** (lire le paramètre depuis le fichier Excel) : un apprenant qui n'a pas compris que `Excel.Workbook` rend une table (et non une valeur) passe 30 minutes à comprendre pourquoi `MoisCible = "MoisCible"` au lieu de `MoisCible = 4`.

## 11. Exercices autonomes

- **Exercice 11.1 (15 min).** Dans la requête `mars2025_v1` du §6, remplacer l'étape `Types_texte` par une étape qui ne force **pas** le type `montant_ttc` en text (le laisser en Any). Actualiser la requête et noter le **nombre d'erreurs** dans la colonne `montant_ttc`. La règle du module est toujours typer explicitement — c'est le contraire de l'intuition (typer augmente les erreurs mais les rend visibles).

- **Exercice 11.2 (30 min).** La requête `mars2025_v1` produit 14 étapes. **Remplacer** l'étape `Tri` par un tri sur `id_vente` seul (au lieu de `date_vente` puis `id_vente`). Recharger et vérifier que le **dédoublonnage retire toujours 86 lignes** (mesure `m05_filrouge_mars_doublons`). L'enseignement : le tri précède le dédoublonnage, mais l'ordre des colonnes dans le tri n'a pas d'effet sur le dédoublonnage par clé métier (la clé est inchangée).
- **Exercice 11.3 (45 min).** Sur le brut complet (243 360 lignes), créer la requête `pivot_brut_v1` qui pivote par `cat` (proxy = 1<sup>re</sup> lettre de `id_produit`) × mois. Vérifier : **12 lignes × 9 colonnes** (mesures `m05_brut_pivot_lignes` × `m05_brut_pivot_colonnes`), **total 15 639 751 286 FCFA** (mesure `m05_brut_total_pivot`). Dépivoter et vérifier : **108 lignes** (mesure `m05_brut_depivot_lignes`).
- **Exercice 11.4 (60 min, optionnel).** Mesurer le temps d'actualisation de la requête `mars2025_v1` sur le brut complet (243 360 lignes). Répéter 5 fois, calculer la moyenne et l'écart-type. Comparer aux temps des requêtes SQL DuckDB (C03) et pandas (C04) sur le même fichier. **Étiqueter** le résultat comme « estimation, à vérifier sur un fichier plus gros ».

## 12. Correction détaillée

- **Exercice 11.1.** Sans le typage explicite, Power Query met `montant_ttc` en Any. L'étape `Compteur_erreurs_conversion` rend 114 (mesure `m05_filrouge_mars_montants_texte`). La leçon est contre-intuitive : typer explicitement met les erreurs en évidence, mais en convertit aussi certaines en Error. La règle est toujours typer — le compteur d'erreur est l'assurance qualité.
- **Exercice 11.2.** Le dédoublonnage retire **toujours 86 lignes** (mesure `m05_filrouge_mars_doublons`), quel que soit l'ordre du tri. La raison : la clé 5-colonnes est indépendante de l'ordre des lignes, et `Table.Distinct` garde la première occurrence — qui est la même quel que soit l'ordre (les doublons sont copiés en fin de fichier).
- **Exercice 11.3.** Le pivot produit bien **12 lignes × 9 colonnes** (mesures `m05_brut_pivot_lignes` × `m05_brut_pivot_colonnes`), total 15 639 751 286 FCFA (mesure `m05_brut_total_pivot`). Le dépivot produit 108 lignes (mesure `m05_brut_depivot_lignes`), et la **somme de la colonne CA dépivotée est identique au total du pivot** : c'est la preuve que pivot et dépivot sont inverses. L'erreur classique est de dépivoter `id_vente` au lieu de `cat` : on obtient une table qui multiplie chaque ligne par le nombre de catégories (9), ce qui n'a aucun sens — c'est l'objet du piège n°4 du §8.
- **Exercice 11.4.** Les temps dépendent de la machine ; sur l'atelier de référence (Excel Microsoft 365, machine virtuelle Linux 4 Go), les **mesures** publiées en C05 sont : 47 s Power Query (mars), ≈ 27 min Power Query (brut complet). Note : ces valeurs ne sont pas des clés `m05_*` ; elles sont étiquetées « estimation, à vérifier ».

## 13. Mini-projet M05.P2 — « La requête paramétrée sur le projet M05.P » (1 h 30)

**Énoncé.** Reprendre la requête `mars2025_v1` du §6 et l'appliquer au **vrai fichier du projet** (03\_exercices/dossier\_M05/ventes\_2023\_2024.csv, 121 720 lignes). Le **livrable** est une requête Power Query `projet2025_v1` qui : - Coupe à la date sur les **120 000 premières lignes** du fichier (la fenêtre du projet). - Paramètre le mois cible (`MoisCible` = 3 pour mars 2025, etc.). - Agrège par client × mois. - Exporte en « **Connexion uniquement** » (pas de duplication dans le classeur).

### Critères de réussite.

1. **6 884 lignes** dans la sortie (mars 2025 coupé sur la fenêtre).
2. **Total montant\_ttc = 464 096 003 FCFA** (mesure `m05_filrouge_mars_total_ttc`).

3. **493 dates transposées réparées** (mesure `m05_fenetre_dates_transposees`).
4. **Compteur d'erreur de conversion affiché** dans la requête (sa valeur n'est pas jugée, sa présence oui).
5. La requête est en « **Connexion uniquement** » (vérifiable dans le volet « Requetes du classeur »).
6. **Aucune valeur de `montant_ttc` négative** n'a été filtrée silencieusement (les retours sont conservés).

**Barème (/10).** 2 points par critère sauf le 5 (« Connexion uniquement ») qui vaut 2 points, le tout sommant 10.

## 14. Boîte à outils du chapitre

**Boîte à outils.** Sept objets, un seul but — reproduire la requête `mars2025_v1` dans Excel :

1. **La table des 10 opérations en Power Query** (§5) — l'application Power Query de la table du C01. À imprimer et à garder sous le coude.
2. **La requête `mars2025_v1` complète** (§6) — les 14 étapes du fil rouge, en code M copiable.
3. **Le paramètre `MoisCible`** (§5.13) — la robustesse au fichier du mois suivant.
4. **Le Health Check en 4 questions** (§5.14) — le diagnostic rapide d'une requête qui ne fait pas ce qu'elle devrait.
5. **Le compteur d'erreur** (§5.3) — le réflexe à installer après chaque conversion.
6. **Le pivot/dépivot** (§5.11) — l'opération de résumé par catégorie et mois, qui alimente le tableau de bord du C05.
7. **Le verdict publié en clé `m05_*`** (§7) — chaque chiffre cité doit apparaître dans `chiffres_cites.json` ; sinon, c'est une promesse creuse.

## 15. Résumé du chapitre

- **Power Query** est l'**ETL du pauvre** : gratuit, déjà installé (Excel), parlant la même langue que les collègues non techniques. Son défaut est d'être **non exécutable dans cet atelier** : chaque bloc M publié est une **recette** (règle 7 du §3 du plan M05), et chaque chiffre est mesuré en SQL/pandas.
- **La requête `mars2025_v1` a 14 étapes** : Source → Sélectionner → Convertir (texte puis nombre) → Paramètre `MoisCible` → Coupe → Tri → Dédoublonnage (clé 5-colonnes) → Réparation (transposition) → Fusion (clé complète) → Agrégation → Sortie. La requête tient en 30 lignes de code M.
- **Le paramètre `MoisCible`** est la clef de voûte de la robustesse : la même requête produit mars, avril ou mai sans réécrire le code. Sans paramètre, chaque mois nécessite de réouvrir la requête et modifier la date.
- **Le pivot/dépivot** sur le brut complet produit **12 lignes × 9 colonnes** (pivot) ou **108 lignes** (dépivot), pour un total de **15 639 751 286 FCFA**. C'est le tableau de bord par catégorie × mois du C05.
- **Le verdict** reste celui du C01 : 6 884 lignes, 464 096 003 FCFA, 114 montants texte, 86 doublons, 32 remises > 1, 74 retours, 10 inconnus, 101 enrichies, 3 978 naïves. **Rejouable** sur avril (6 639) et mai (5 530) sans modification du pipeline.

## 16. À retenir

**À retenir.** Power Query est un **moteur de transformation** qui vit dans Excel. Il **reproduit le pipeline à l'ouverture du classeur** : c'est la différence entre un classeur qui marche par chance et un classeur qui marche par construction. La requête paramétrée est la clef de voûte de la robustesse : sans paramètre, le pipeline marche pour un mois ; avec paramètre, il marche pour tous les mois. Chaque bloc M publié est une **recette**, et chaque chiffre est mesuré en SQL/pandas — c'est ce qui rend le module honnête.

**À retenir.** Les 4 pièges classiques de Power Query (résumé) : (a) Encoding non spécifié → BOM dans la première colonne ; (b) type Any → tri impossible, Distinct trop agressif ; (c) Join-Kind.LeftAnti par défaut → la jointure garde les **mauvaises** lignes ; (d) Unpivot sur un identifiant → la table explose. La règle unique : toujours expliciter l'option par défaut.

## 17. Évaluation formative (auto-correction, 8 min)

**Question 1.** Citez les 14 étapes de la requête `mars2025_v1` du §6 dans l'ordre. (Réponse : §6.)

**Question 2.** Qu'est-ce qu'un **paramètre** dans Power Query ? À quoi sert le paramètre `MoisCible` ? (Réponse : une variable nommée, lue dans un fichier externe ou saisie par l'utilisateur. Le paramètre `MoisCible` permet de produire mars, avril ou mai 2025 sans réécrire la requête.)

**Question 3.** Pourquoi la requête `mars2025_v1` utilise-t-elle **deux étapes** pour la conversion de `montant_ttc` (texte puis nombre) ? (Réponse : la première étape (`Montant_nettoye`) trace la transformation (retrait du suffixe « FCFA » et des espaces), la seconde (`Montant_converti`) donne le type final (`Int64.Type`). Volontairement deux étapes : la première pour la traçabilité, la seconde pour la performance.)

**Question 4.** Citez les 4 pièges classiques de Power Query. (Réponse : §8, encadré « À retenir ».)

**Question 5.** Sur mars 2025, combien de dates transposées sont réparées par l'étape `Mois_repare` ? (Réponse : 23 lignes, mesure `m05_filrouge_mars_dates_transposees`.)

**Question 6.** Qu'est-ce que le **Health Check** en 4 questions ? Citez-les dans l'ordre. (Réponse : §5.14. (1) Le type est-il bon ? (2) L'étape de filtre est-elle correcte ? (3) La fusion a-t-elle développé les bonnes colonnes ? (4) L'agrégation utilise-t-elle la bonne fonction ?)

**Question 7.** Pourquoi le pivot sur le brut complet produit-il **12 lignes × 9 colonnes** ? Quel est le proxy catégorie utilisé ? (Réponse : le pivot est par mois (12 valeurs distinctes) × catégorie proxy (9 valeurs distinctes, la 1<sup>re</sup> lettre de `id_produit`). C'est un proxy, pas la vraie catégorie de produit — la vraie catégorie est dans `produits.csv` mais n'est pas jointe ici. C'est un objet du C05.)